基于边缘AI与联盟链的监控视频防篡改解决方案

📋 **各模块技术选型记录（头脑风暴）**

层级	模块	选型决策
L1 边缘智能层	GOP 切分	MVP 先用思路 A：FFmpeg/pyav 实时解码切分（通用性优先），后续可升级为思路 B：RTP 字节级切分（性能优先，需适配不同厂商流格式）
L1 边缘智能层	密码学哈希	SHA-256，对 GOP 原始编码字节计算
L1 边缘智能层	感知哈希	MVP 阶段先用传统 pHash（imagehash 库），后续可升级为深度方案（MobileNetV3 + LSH）
L1 边缘智能层	语义指纹	YOLOv8-nano 提取目标类别计数，首次生成后固化存储，SHA-256(JSON) 纳入 Merkle 叶子。存储：MVP 先用 B（分布式存储 IPFS/MinIO），后续改进为 C（边缘本地 + 分布式存储双写）
L1 边缘智能层	Merkle 树	三级结构（GOP → Chunk 30s → Segment 5min），叶子 = SHA-256(crypto_hash ∥ perceptual_hash ∥ semantic_hash)，自己实现（支持增量追加），树结构序列化后存 IPFS/MinIO
L1 边缘智能层	自适应锚定（EIS）	轻量版：复用 YOLO 目标数量作为 EIS，滑动窗口取中位数平滑（抗误检），状态切换需连续确认（快升慢降防抖）。完整版（光流 + 异常检测 + 规则引擎）列为后续优化

层级	模块	选型决策
L2 聚合网关层	聚合网关	FastAPI 后端服务，SQLite 存储历史数据和 Merkle 树结构，每 30s 构建 EpochMerkleTree。演示：1 路真实视频 + 2~3 路模拟数据。设备签名 MVP 不做，后续可加设备私钥签名机制
L3 联盟链层	链 + 智能合约	Hyperledger Fabric（已部署），单机 Docker 模拟多节点。MVP 只做 AnchorContract（存证）+ VerifyContract（验证），AuditContract 留后续。三态判定放链下，链上只做 Merkle 验证（INTACT / NOT_INTACT）
L0 分布式存储层	分布式存储	MVP 用 MinIO（Docker 单容器部署，Python minio SDK），自行对文件内容算 SHA-256 作为 CID 实现内容寻址。存储内容：视频分片 + 语义 JSON + Merkle 树结构 + Epoch 记录。后续可迁移到 IPFS 集群实现去中心化存储

🚀 **MVP 开发计划（预计 8~9 天）**

Phase 1：最小端到端闭环（3~4 天）

目标：一段录制视频 → 按 GOP 切分 → 算哈希 → 存到 MinIO → 上链 → 能验证

▼ Step 1：GOP 切分 + SHA-256（半天）✅

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (SecureLens)
【操作】新建文件 services/gop_splitter.py
现有项目的视频处理在 services/detection_service.py，用 cv2.VideoCapture 逐帧读取 + YOLO 检测。
现在需要新增一个并行的 GOP 级别切分模块，不修改现有检测流程。

技术要求：
1. 使用 pyav (av) 库读取输入的 mp4 视频文件
```

```
2. 遍历视频流的每一个 packet，通过检测关键帧（packet.is_keyfram
e）来识别 GOP 边界
3. 每当遇到新的关键帧，将之前累积的所有 packet 的原始字节拼接为一个
GOP
4. 对每个 GOP 的原始编码字节（不是解码后的像素）计算 SHA-256 哈希
5. 同时记录每个 GOP 的元数据：gop_id、起始时间戳、结束时间戳、帧
数、字节大小

接口设计：
- split_gops(video_path: str) -> List[GOPData]
- GOPData 是一个 dataclass，包含：
  - gop_id: int
  - raw_bytes: bytes  # GOP 原始编码数据
  - sha256_hash: str   # 十六进制字符串
  - start_time: float  # 秒
  - end_time: float    # 秒
  - frame_count: int
  - byte_size: int
  - keyframe_bytes: bytes  # I 帧解码后的图像数据（供后续 pHas
h 和 YOLO 使用）

写一个 main 函数，读取 sample_videos/ 目录下的 mp4 文件，调用 sp
lit_gops，
打印每个 GOP 的 gop_id、时间范围、字节大小、SHA-256 哈希值。

依赖：pip install av
```

测试命令：

```
# 离线测试
python -m services.gop_splitter --file sample_videos/test.
mp4
```

▼ Step 2：MinIO 存储（2~3h）✅

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (Sec
ureLens)
【操作】新建 services/minio_storage.py + 修改 config.py
现有项目用本地文件系统存储证据（evidences/ 目录）。
现在新增 MinIO 分布式存储，与本地存储并存，不删除现有本地存储逻辑。

前置条件：
- MinIO 已通过 Docker 启动：
  docker run -p 9000:9000 -p 9001:9001 minio/minio server
/data --console-address ":9001"
- 默认凭证：minioadmin / minioadmin

技术要求：
1. 使用 minio Python SDK
2. 初始化时自动创建 bucket "video-evidence"（如不存在）
3. 实现上传 GOP 分片：
   - 对文件内容算 SHA-256 作为 CID
   - 存储路径格式：{device_id}/segment_{timestamp}/{cid}.h26
4
   - 将 gop_id、时间戳、sha256_hash 存为对象的元数据（metadata）
4. 实现下载 GOP 分片（按 CID 检索）
5. 实现列出某个设备某个时间范围内的所有 GOP

接口设计：
- VideoStorage 类：
  - __init__(endpoint, access_key, secret_key, bucket_nam
e)
  - upload_gop(device_id: str, gop: GOPData) -> str  # 返回
CID
  - download_gop(device_id: str, cid: str) -> bytes
  - list_gops(device_id: str, start_time: str, end_time: s
tr) -> List[dict]
  - upload_json(device_id: str, filename: str, data: dict)
-> str  # 上传 JSON 文件
  - download_json(device_id: str, filename: str) -> dict
```

```
写一个测试脚本，调用 Step 1 的 split_gops 切分视频，然后将所有 GOP 上传到 MinIO，
再下载并验证 SHA-256 一致性。

依赖：pip install minio
```

测试命令：

```
# 确保 MinIO Docker 已运行
docker run -d -p 9000:9000 -p 9001:9001 minio/minio server /data --console-address ":9001"
# 运行测试
python test_minio_storage.py --file sample_videos/test.mp4
```

▼ Step 3：简易 Merkle 树（半天）✅

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (SecureLens)
【操作】修改 services/merkle_utils.py，增加序列化和类封装
现有文件 services/merkle_utils.py 已有：
- sha256_digest(data: bytes) -> bytes
- build_merkle_root_and_proofs(leaf_hashes: List[str]) -> Tuple[str, List[List[Dict]]]
- apply_merkle_proof(leaf_hash: str, proof: List[Dict]) -> str
这些函数已被 detection_service.py 和 chaincode 使用，不能删改。

技术要求：
1. 自己实现，不用第三方库
2. 哈希算法用 SHA-256
3. 叶子节点数量不足 2 的幂次时，用最后一个叶子重复填充 (padding)
4. 支持生成 Merkle Proof (从叶子到根的路径)
5. 支持验证 Merkle Proof (给定叶子哈希 + proof + root，返回 bool)
```

```
6. 支持序列化为 JSON（完整树结构）和反序列化

接口设计：
- MerkleTree 类：
  - __init__(leaves: List[str])  # leaves 是十六进制哈希字符串列表
  - root: str  # 根哈希
  - get_proof(leaf_index: int) -> List[dict]  # 每个元素包含 {"hash": str, "position": "left"|"right"}
  - @staticmethod verify_proof(leaf_hash: str, proof: List[dict], root: str) -> bool
  - to_json() -> str
  - @classmethod from_json(cls, json_str: str) -> MerkleTree

写单元测试：
- 测试 4 个叶子的树，验证每个叶子的 proof 都能通过
- 测试 5 个叶子（非 2 的幂次）的 padding 逻辑
- 测试篡改一个叶子后 proof 验证失败
- 测试序列化/反序列化往返一致

然后集成测试：用 Step 1 的 GOP SHA-256 哈希列表构建 Merkle 树，打印 root，
并对其中一个 GOP 生成并验证 proof。
```

测试命令：

```
# 单元测试
pytest tests/test_merkle_utils.py -v
# 集成测试
python test_merkle_tree_integration.py
```

▼ Step 4：Fabric 上链（1 天）✅

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (SecureLens)
【操作】修改 chaincode/chaincode.go + 修改 services/fabric_client.py
现有 chaincode/chaincode.go 已有：
- CreateEvidence / CreateEvidenceBatch / ReadEvidence / VerifyEvidence / VerifyEvent
- 私有数据集合 (PDC)、工单管理、审计轨迹等
这些现有功能保持不变。
现有 services/fabric_client.py 已有：
- build_fabric_env(fabric_samples) -> (env, orderer_ca, org2_tls)
- invoke_chaincode(env, orderer_ca, org2_tls, channel, chaincode, function, args)
- query_chaincode(env, channel, chaincode, function, args)
- get_latest_block_number(env, channel)
这些函数保持不变，新功能复用它们。

== Chaincode 部分 (Go) ==
创建 chaincode/anchor/anchor.go，实现 AnchorContract：

功能：
1. Anchor(epochId string, merkleRoot string, timestamp string, deviceCount int)
   - 将 epochId 作为 key，存储 {merkleRoot, timestamp, deviceCount} 到链上状态数据库
   - 要求 timestamp 必须大于该 gateway 上一次提交的 timestamp (防时间回退)
   - 触发事件 AnchorEvent
2. QueryAnchor(epochId string) -> AnchorRecord
   - 按 epochId 查询已存储的键值
3. QueryAnchorsByRange(startEpochId string, endEpochId string) -> []AnchorRecord
   - 范围查询
```

```
数据结构：
- AnchorRecord: {EpochId, MerkleRoot, Timestamp, DeviceCou
nt, GatewayId}

== Python 客户端部分 ==
创建 fabric_client/client.py，封装与 Fabric 网络的交互：

- FabricClient 类：
  - submit_anchor(epoch_id: str, merkle_root: str, timesta
mp: str, device_count: int)
  - query_anchor(epoch_id: str) -> dict
  - query_anchors_by_range(start_id: str, end_id: str) ->
List[dict]

使用现有的 Fabric SDK 或 CLI 方式调用。
如果用 CLI，用 subprocess 调用 peer chaincode invoke/query 命
令。

集成测试：
- Step 1 切分视频 → Step 3 构建 Merkle 树 → 拿到 MerkleRoot
- 调用 submit_anchor 提交到链上
- 调用 query_anchor 查询并验证返回的 MerkleRoot 一致
```

测试命令：

```
# 编译 chaincode
cd chaincode && go build ./...

# Go 单元测试
cd chaincode && go test -v ./...

# 集成测试（需要 Fabric test-network 运行中）
python -m pytest tests/test_anchor_integration.py -v
```

▼ Step 5：验证流程（1 天）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (SecureLens)
【操作】修改 chaincode/chaincode.go + 新建 services/gop_verifier.py
现有 chaincode 已有 VerifyEvent（基于事件的 Merkle 验证）。
新增一个针对 GOP/Anchor 的验证方法（不修改现有 VerifyEvent）。

== Chaincode 部分（Go）==
在 chaincode/anchor/anchor.go 中添加 Verify 方法：

Verify(epochId string, leafHash string, proofJSON string)
- 从链上读取 epochId 对应的 merkleRoot
- 解析 proofJSON（Merkle Proof 路径）
- 在链上重新计算：从 leafHash 沿着 proof 逐层哈希，最终得到 computedRoot
- 比较 computedRoot == storedMerkleRoot
- 返回 {"status": "INTACT"} 或 {"status": "NOT_INTACT"}

注意：Go 中使用 crypto/sha256 计算哈希，确保与 Python 端的 SHA-256 结果一致。
proof 中每个元素的 position 决定拼接顺序：
- position="left": hash(proof_hash + current_hash)
- position="right": hash(current_hash + proof_hash)

== Python 验证服务 ==
创建 verification/verifier.py：

Verifier 类：
- verify_gop(device_id: str, epoch_id: str, gop_index: int) -> dict
  流程：
  1. 从 MinIO 下载目标 GOP 的原始字节
  2. 重新计算 SHA-256
  3. 从本地或 MinIO 加载 Merkle 树 JSON，生成该 GOP 的 proof
  4. 调用 Fabric 链上 VerifyContract
```

```
  5. 返回验证结果

集成测试（端到端）：
- 用一段 mp4 视频跑通完整流程：
  切分 → 算哈希 → 存 MinIO → 建 Merkle 树 → 上链 → 验证某个 GOP
→ 返回 INTACT
- 篡改测试：修改 MinIO 中某个 GOP 的字节 → 验证 → 返回 NOT_INTAC
T
```

Phase 2：补全核心创新点（1.5~2 天）

目标：加上三个核心创新，让 demo 有差异化

▼ Step 6：pHash + 三态判定（半天）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (Sec
ureLens)
【操作】新建 services/perceptual_hash.py + 新建 services/tri_
state_verifier.py
       + 修改 services/gop_splitter.py
现有项目已有 Pillow、OpenCV（用于 detection_service.py），可直
接复用。

== edge/perceptual_hash.py ==
技术要求：
1. 使用 imagehash 库和 Pillow
2. 输入：GOPData 中的 keyframe_bytes（I 帧解码后的图像数据）
3. 将 keyframe_bytes 转为 PIL Image，然后计算 pHash
4. 返回 64-bit 哈希值（十六进制字符串）

接口：
- compute_phash(keyframe_bytes: bytes, width: int, height:
int) -> str
- hamming_distance(hash1: str, hash2: str) -> int

== verification/tri_state.py ==
```

```
实现链下三态判定逻辑：

- TriStateVerifier 类：
  - __init__(phash_threshold: int = 10)  # 汉明距离阈值
  - verify(original_sha256: str, original_phash: str,
           current_sha256: str, current_phash: str) -> str
    逻辑：
    - sha256 匹配 AND phash 匹配 → "INTACT"
    - sha256 不匹配 AND phash 汉明距离 <= threshold → "RE_EN
CODED"
    - sha256 不匹配 AND phash 汉明距离 > threshold → "TAMPER
ED"

修改 edge/gop_splitter.py，在 GOPData 中增加 phash 字段，
切分时同时计算 SHA-256 和 pHash。

单元测试：
- 同一张图片 → pHash 相同 → INTACT
- 同一张图片经 JPEG 压缩后 → pHash 距离很小 → RE_ENCODED
- 不同图片 → pHash 距离很大 → TAMPERED

依赖：pip install imagehash Pillow
```

▼ Step 7：语义指纹（半天~1 天）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (Sec
ureLens)
【操作】新建 services/semantic_fingerprint.py + 修改 service
s/gop_splitter.py
现有项目已用 ultralytics YOLO（见 detection_service.py），
但现有检测流程是逐帧检测 + 事件聚合，输出是 AggregatedEvent。
现在需要专门针对 GOP 关键帧的轻量级语义提取。

技术要求：
1. 使用 ultralytics 库加载 YOLOv8-nano 预训练模型（yolov8n.p
```

```
t)
2. 输入：GOPData 中的 keyframe_bytes（I 帧解码后的图像）
3. 运行目标检测，统计每个类别的目标数量
4. 生成结构化语义 JSON：
   {
     "gop_id": 10247,
     "timestamp": "2026-03-11T14:23:05Z",
     "objects": {"person": 3, "car": 2, "bicycle": 1},
     "total_count": 6
   }
5. 对该 JSON 字符串计算 SHA-256，作为 semantic_hash
6. 注意：JSON 序列化时必须使用 json.dumps(data, sort_keys=Tru
e, separators=(',', ':'))
   保证确定性（相同输入始终产生相同的 JSON 字符串）

接口：
- SemanticExtractor 类：
  - __init__(model_path: str = "yolov8n.pt", confidence: f
loat = 0.5)
  - extract(gop: GOPData) -> SemanticFingerprint
  - SemanticFingerprint dataclass:
    - gop_id: int
    - timestamp: str
    - objects: dict  # {"person": 3, "car": 2}
    - total_count: int
    - json_str: str       # 确定性序列化的 JSON
    - semantic_hash: str   # SHA-256(json_str)

修改 GOPData，增加 semantic_hash 字段。

修改 Merkle 树叶子构造：
原来叶子 = sha256_hash
现在叶子 = SHA-256(sha256_hash + phash + semantic_hash)
创建一个辅助函数 compute_leaf_hash(sha256_hash, phash, semant
ic_hash) -> str
```

```
语义 JSON 存储到 MinIO，与视频分片同目录。

依赖：pip install ultralytics
```

▼ Step 8：三级 Merkle 树（半天）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (SecureLens)
【操作】修改 services/merkle_utils.py，新增 HierarchicalMerkleTree
现有 services/merkle_utils.py 已有：
- sha256_digest, build_merkle_root_and_proofs, apply_merkle_proof
- Step 3 新增的 MerkleTree 类、compute_leaf_hash
这些全部保留，在文件末尾新增 HierarchicalMerkleTree 类。
注意：现有的 MerkleBatchManager（detection_service.py）用于事件级别批处理，
和新的 HierarchicalMerkleTree 是两套并行系统，互不干扰。

技术要求：
1. 三级结构：
   - 第一级（叶子）：每个 GOP 的 leaf_hash
   - 第二级（Chunk）：每 30 秒内的所有 GOP 叶子构建一棵小 Merkle 树，得到 ChunkRoot
   - 第三级（Segment）：每 5 分钟内的所有 ChunkRoot 构建上层 Merkle 树，得到 SegmentRoot

2. 支持增量追加：
   - add_gop(leaf_hash: str, timestamp: float)
     根据 timestamp 自动判断当前属于哪个 Chunk（每 30s 一个）
     当前 Chunk 满30s 时自动封闭，开新 Chunk
   - close_segment() -> str
     封闭当前 Segment，返回 SegmentRoot
     每 5 分钟调用一次，或在视频结束时调用
```

```
3. 精确篡改定位：
   - get_full_proof(gop_index: int) -> dict
     返回从叶子到 SegmentRoot 的完整 Merkle 路径：
     {
       "gop_to_chunk_proof": [...],   # GOP 叶子到 ChunkRoo
t
       "chunk_to_segment_proof": [...], # ChunkRoot 到 Seg
mentRoot
       "chunk_index": int,
       "gop_index_in_chunk": int
     }
   - locate_tampered_gop(segment_root: str) -> List[int]
     给定一组重新计算的 GOP 哈希，找出与原始树不一致的 GOP 索引列表

4. 序列化：完整的三级树结构可序列化为 JSON 存储到 MinIO

接口：
- HierarchicalMerkleTree 类，包含以上所有方法
- 保留原来的 MerkleTree 类作为内部构建块

单元测试：
- 模拟 10 分钟的 GOP 流（每 2 秒一个，共 300 个）
- 验证生成了 2 个 Segment，每个包含 10 个 Chunk
- 验证任意 GOP 的 full proof 能通过
- 篡改某个 GOP 后，locate_tampered_gop 能精确定位
```

Phase 3：完善架构层（1.5 天）

目标：补上聚合网关和自适应锚定，让架构完整

▼ Step 9：聚合网关（1 天）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC (Sec
ureLens)
【操作】修改 web_app.py + 新建 services/gateway_service.py
现有 web_app.py 已是 FastAPI 应用，包含：
```

```
- 视频监控首页、工单管理、审计报告、配置页面
- /api/verify/{event_id}、/api/history/{event_id} 等接口
现有 MerkleBatchManager 用于事件级批处理，保持不变。

技术要求：
1. 使用 FastAPI + uvicorn
2. SQLite 存储历史数据（用 sqlite3 标准库即可）

API 接口：

1. POST /report
   接收边缘设备的上报：
   {
     "device_id": "cam_001",
     "segment_root": "a3f8c1...",
     "timestamp": "2026-03-11T14:25:00Z",
     "semantic_summaries": [...],
     "gop_count": 150
   }
   存入 SQLite。

2. GET /epoch/{epoch_id}
   查询某个 epoch 的聚合结果。

3. GET /proof/{epoch_id}/{device_id}
   返回某台设备在 EpochMerkleTree 中的 Merkle Proof。

后台定时任务（每 30 秒）：
- 收集近 30 秒内所有设备上报的 SegmentRoot
- 以各设备的 SegmentRoot 作为叶子，构建 EpochMerkleTree
- 得到 EpochRoot
- 调用 FabricClient.submit_anchor 提交上链
- 保存 EpochMerkleTree 结构到 SQLite

使用 APScheduler 或 asyncio 实现定时任务。
```

```
模拟脚本 gateway/simulate_devices.py：
- 模拟 3 台设备（cam_001, cam_002, cam_003）
- 每台设备每隔几秒向网关 POST /report 发送模拟数据
- 其中 cam_001 用真实视频切分结果，cam_002 和 cam_003 用随机生成的哈希

依赖：pip install fastapi uvicorn apscheduler
```

▼ Step 10：轻量版 EIS（2~3h）

```
【现有项目】https://github.com/NgokzittYu/CCTV-W-FABRIC（SecureLens）
【操作】新建 services/adaptive_anchor.py
现有项目的 event_aggregator.py 做的是多帧事件聚合（IoU 跟踪），新的 AdaptiveAnchor 是完全不同的模块，不修改 event_aggregator.py。

技术要求：
1. EIS 数据源：复用 SemanticFingerprint 中的 total_count（YOLO 目标数量）
2. 滑动窗口平滑：保留最近 10 个 GOP 的 total_count，取中位数
3. EIS 计算：
   - smoothed_count == 0 → EIS = 0.1
   - smoothed_count <= 5 → EIS = 0.5
   - smoothed_count > 5  → EIS = 0.9
4. 状态切换防抖：
   - 三个等级：LOW（EIS < 0.3），MEDIUM（0.3-0.7），HIGH（EIS > 0.7）
   - 升级需连续 3 次确认（快升）
   - 降级需连续 5 次确认（慢降）
5. 不同等级对应不同的上报频率：
   - LOW: 每 5 分钟上报一次 SegmentRoot
   - MEDIUM: 每 1 分钟上报一次
   - HIGH: 每 10 秒上报一次
```

```
接口：
- AdaptiveAnchor 类：
  - __init__(window_size=10, upgrade_confirm=3, downgrade_
confirm=5)
  - update(semantic: SemanticFingerprint) -> AnchorDecisio
n
  - AnchorDecision dataclass:
    - eis_score: float
    - smoothed_count: int
    - level: str  # "LOW" | "MEDIUM" | "HIGH"
    - report_interval_seconds: int  # 300 | 60 | 10
    - should_report_now: bool  # 根据距上次上报的时间判断

单元测试：
- 模拟一组 total_count 序列 [0,0,0,0,0, 3,3,3,3,3, 8,8,8,8,
8]
- 验证等级从 LOW → MEDIUM → HIGH 的切换时机
- 模拟偶发异常 [3,3,3,15,3,3,3,3,3,3]，验证中位数不受影响
- 验证快升慢降：升级 3 次确认，降级 5 次确认
```

Phase 4：演示和打磨（1.5 天）

目标：演示效果拉满

▼ Step 11：篡改演示脚本（半天）

```
【现有项目】SecureLens 已有 verify_evidence.py（CLI 验证脚本）
和 services/ 下的完整服务层。
【操作】新建 scripts/tamper_demo.py，复用现有服务层，实现端到端篡
改检测演示。
请基于现有项目创建 scripts/tamper_demo.py，实现端到端的篡改检测演
示。

技术要求：
准备三种测试场景，每种场景完整跑通 “存证 → 篡改/转码 → 验证” 流程：
```

```
场景 1：INTACT（完整）
- 输入视频 → 切分 → 算哈希 → 存 MinIO → 建树 → 上链
- 直接验证原始视频 → 期望结果 INTACT

场景 2：RE_ENCODED（转码未篡改）
- 存证同场景 1
- 用 FFmpeg 将原始视频转码（如改码率、改编码格式）：
  ffmpeg -i input.mp4 -b:v 500k -c:v libx264 re_encoded.mp
4
- 对转码后的视频重新切分和算哈希
- 验证 → SHA-256 不匹配，pHash 匹配 → 期望结果 RE_ENCODED

场景 3：TAMPERED（已篡改 + 精确定位）
- 存证同场景 1
- 用代码篡改其中第 N 个 GOP：
  - 方法：用另一段视频的某个 GOP 替换掉目标 GOP
  - 或者简单地修改 GOP 原始字节的几个 byte
- 验证 → SHA-256 不匹配，pHash 不匹配 → 期望结果 TAMPERED
- 调用 locate_tampered_gop 定位到具体 GOP 索引
- 输出："篡改发生在 GOP #N，时间范围 XX:XX - XX:XX"

每个场景用彩色 print 输出清晰的流程日志，便于答辩演示。
可用 rich 库美化终端输出。

依赖：pip install rich
```

▼ Step 12：简易 Web 界面（1 天，可选）

```
【现有项目】SecureLens 已有 web_app.py（FastAPI + Jinja2）和 t
emplates/ 目录（index.html, workorder.html, audit.html, con
fig.html）。
【操作】修改 web_app.py 添加 GOP 验证路由 + 新建 templates/gop_
verify.html 可视化页面。不使用 Streamlit，沿用现有 FastAPI + Ji
nja2 架构。
请基于现有 FastAPI 项目扩展 Web 界面，添加 GOP 验证可视化功能。
```

```
技术要求：
1. 使用 Streamlit（最快的 Python Web UI 方案）
2. 页面布局：

   左侧边栏：
   - 系统状态（Fabric 连接、MinIO 连接、设备数量）
   - 模式选择：存证模式 / 验证模式

   存证模式页面：
   - 上传视频文件（st.file_uploader）
   - 点击"开始存证" → 显示进度条
   - 实时显示：GOP 切分数量、哈希计算进度、Merkle Root、链上交易 ID
   - 显示语义指纹示例（某个 GOP 的 YOLO 检测结果 + 关键帧图片）

   验证模式页面：
   - 上传待验证的视频文件
   - 选择原始存证的 epoch_id
   - 点击"开始验证" → 显示进度
   - 结果展示：
     - 大字显示三态结果（绿色 INTACT / 黄色 RE_ENCODED / 红色 TAMPERED）
     - 如果是 TAMPERED，显示篡改定位：
       - 时间线可视化，标红被篡改的 GOP 位置
       - 显示篡改时间范围
     - 显示原始 vs 当前的哈希对比表
     - 显示 Merkle Proof 路径可视化

3. 后端调用：直接 import 已有的 Python 模块（gop_splitter, merkle_tree,
   fabric_client, minio_client, tri_state 等）

运行：streamlit run demo/web_app.py
```

```
依赖：pip install streamlit
```

一、总体架构

本方案采用 **四层架构**：边缘智能层 → 聚合网关层 → 联盟链层 → 分布式存储层。

系统总体架构图

```
flowchart TB
    subgraph L4["验证与应用层"]
        A1["司法取证终端"]
        A2["安防管理平台"]
        A3["第三方审计接口"]
    end

    subgraph L3["联盟链层 (Layer 3)"]
        C1["存证合约<br>Anchor()"]
        C2["验证合约<br>Verify()"]
        C3["审计合约<br>TamperAlert()"]
        CD["链上数据：EpochRoot / 语义摘要 / 设备ID / 时间戳"]
    end

    subgraph L2["聚合网关层 (Layer 2)"]
        GW["收集多台边缘设备的 SegmentRoot<br>→ 构建 EpochMerkleT
ree<br>批量聚合，降低上链频次（如每30s提交一次）"]
    end

    subgraph L1["边缘智能层 (Layer 1)"]
        CAM["摄像头"] --> BOX["边缘AI盒子 (Edge AI Box)"]
        BOX --> B1["① GOP切分 & 双哈希计算"]
        BOX --> B2["② AI语义指纹提取"]
        BOX --> B3["③ 分层Merkle树构建"]
        BOX --> B4["④ 自适应锚定频率决策"]
    end
```

```
    subgraph L0["分布式存储层 (Layer 0)"]
        IPFS["IPFS / MinIO 集群（内容寻址存储）<br>视频分片存储，CID与链上记录关联"]
    end

    L4 -->|"查询 & 验证"| L3
    L3 -->|"批量提交 EpochRoot"| L3
    GW -->|"批量提交 EpochRoot"| L3
    L1 -->|"上报 SegmentRoot + 语义摘要"| L2
    L1 -->|"视频流写入"| L0

    style L4 fill:#1a1a2e,stroke:#e94560,color:#fff
    style L3 fill:#16213e,stroke:#e94560,color:#fff
    style L2 fill:#0f3460,stroke:#e94560,color:#fff
    style L1 fill:#533483,stroke:#e94560,color:#fff
    style L0 fill:#2b2d42,stroke:#e94560,color:#fff
```

存证流程图

```
flowchart LR
    A["摄像头"] -->|"视频流"| B["边缘AI盒子"]
    B --> C["按GOP切分视频流"]
    C --> D["对每个GOP计算<br>SHA-256 + 感知哈希"]
    D --> E["运行YOLOv8-nano<br>提取语义摘要"]
    E --> F["计算EIS<br>决定锚定频率"]
    F --> G["构建三级Merkle树"]
    G --> H["视频分片写入IPFS<br>获取CID"]
    H --> I["上报SegmentRoot<br>+ 语义摘要 + CID映射"]
    I --> J["聚合网关<br>聚合为EpochRoot"]
    J --> K["提交上链<br>→ 联盟链"]
```

验证流程图

```
flowchart LR
    A["验证请求方<br>指定设备ID+时间范围"] --> B["验证服务"]
    B --> C["从IPFS拉取<br>原始视频分片"]
    C --> D["重新计算目标GOP<br>的双哈希"]
    D --> E["从聚合网关<br>获取Merkle路径"]
    E --> F["调用链上<br>VerifyContract"]
    F --> G{"三态验证结果"}
    G -->|"INTACT"| H["✅ 完整"]
    G -->|"RE-ENCODED"| I["⚠ 转码但内容未变"]
    G -->|"TAMPERED"| J["❌ 已篡改<br>定位到具体GOP"]
```

二、边缘智能层：视频 DNA 的生成

这是整个方案的核心引擎，部署在靠近摄像头的边缘 AI 盒子上（如 NVIDIA Jetson 系列），负责将原始视频流转化为紧凑、多维、可验证的"视频 DNA"。

2.1 视频流切分

视频流按照 H.264/H.265 编码结构，以 **GOP（Group of Pictures）** 为最小处理单元进行切分。一个典型的 GOP 包含一个 I 帧和若干 P/B 帧，时长通常为 1~2 秒。选择 GOP 作为最小粒度而非单帧，是因为 GOP 是编码上独立可解码的最小单元，在粒度和效率之间取得了合理平衡。

2.2 核心创新一：双哈希锚定机制（Dual-Hash Anchoring）

对每个 GOP，**同时计算两种性质截然不同的哈希值**：

(A) 密码学哈希（Cryptographic Hash） ——对 GOP 原始二进制数据计算 SHA-256。这是一种"脆弱指纹"，任何一个比特的改变都会导致哈希值完全不同。它的作用是检测任何层面的修改，精度极高。

(B) AI 感知哈希（Perceptual Hash） ——利用轻量级卷积神经网络（如 MobileNetV3 的特征提取层）对 GOP 关键帧进行推理，提取一个 256 维的深度特征向量，再通过局部敏感哈希（LSH）压缩为一个 64-bit 的紧凑指纹。这是一种"鲁棒指纹"，它对画面内容敏感，但对编码格式转换、合理压缩等操作具有容忍度。

双哈希协同验证的价值在于：单一的密码学哈希虽然精确，但在实际场景中，视频可能因存储系统迁移而被重新编码（如从 H.264 转为 H.265），此时密码学哈希必然改变，但

这不意味着内容被篡改。引入感知哈希后，系统可以区分以下三种状态：

密码学哈希	感知哈希	判定结果
✅ 一致	✅ 一致	**完全未修改**（原始比特流完好）
❌ 不一致	✅ 一致	**格式变更但内容未篡改**（如转码、压缩）
❌ 不一致	❌ 不一致	**内容已被篡改**（需告警）

这种三态判定能力是传统单一哈希方案所不具备的，显著降低了误报率，提升了方案的实用性。

2.3 核心创新二：AI 语义指纹（Semantic Fingerprint）

边缘 AI 在计算双哈希的同时，对 GOP 关键帧运行轻量级目标检测模型（如 YOLOv8-nano），提取一段结构化的**语义摘要**：

```
{
  "gop_id": 10247,
  "timestamp": "2026-03-11T14:23:05Z",
  "objects": {"person": 3, "car": 2, "bicycle": 1},
  "scene_type": "intersection",
  "motion_intensity": 0.72,
  "anomaly_score": 0.15
}
```

语义指纹的作用是为视频增加一个**内容语义层面的校验维度**。即便在极端理论假设下攻击者构造了一段哈希碰撞的替代视频（实际上对 SHA-256 不可行），其语义内容也几乎不可能与原始语义摘要吻合。更重要的是，语义摘要在司法场景中具有直观的辅助说明价值——法官可以看到链上记录显示"该时段画面中有 3 人 2 车"，与视频画面相互印证。

2.4 核心创新三：分层 Merkle 树与精确篡改定位

传统方案通常对整段视频（如一小时）计算一个哈希值上链。这种方式的问题是：一旦验证失败，只知道"这段视频被改了"，但无法定位具体是哪一秒被篡改。

本方案构建**三级分层 Merkle 树**：

```
SegmentRoot (每 5 分钟)
        /                \
```

```
ChunkRoot₁     ChunkRoot₂ ...  (每 30 秒)
    / \            / \
GOP_H₁ GOP_H₂  GOP_H₅ GOP_H₆ ... (每 1~2 秒)
```

- **第一级（叶子节点）**：每个 GOP 的双哈希值（SHA-256 // PerceptualHash）作为叶子。
- **第二级（Chunk）**：每 30 秒内的所有 GOP 叶子构建一棵子 Merkle 树，得到 ChunkRoot。
- **第三级（Segment）**：每 5 分钟内的所有 ChunkRoot 再构建上层 Merkle 树，得到 SegmentRoot。

SegmentRoot 是上报给聚合网关的核心数据。当需要验证时，可以利用 Merkle 证明路径（Merkle Proof）以 **O(log n)** 的复杂度快速定位到被篡改的具体 GOP，即精确到 1~2 秒的时间范围。

完整的 Merkle 树结构存储在边缘设备和分布式存储中，链上只存储 SegmentRoot，实现了**极致的存储压缩**。

2.5 核心创新四：AI 驱动的自适应锚定策略（Adaptive Anchoring）

并非所有时段的监控画面都同等重要。深夜空无一人的走廊与白天发生冲突的街头，对防篡改的精度要求截然不同。本方案利用边缘 AI 的场景理解能力，**动态调整上链的粒度和频率**：

边缘 AI 持续计算每个 GOP 的**事件重要性分数（Event Importance Score, EIS）**，EIS 综合考量以下因素：运动强度（光流分析）、目标数量与类型（检测到人/车的数量）、异常检测分数（偏离正常模式的程度）、预设规则触发（如越界检测、徘徊检测）。

基于 EIS，系统自动切换锚定策略：

EIS 区间	场景示例	锚定策略
0.0 ~ 0.3	深夜空旷走廊	低频模式：每5分钟上报一次 SegmentRoot
0.3 ~ 0.7	正常人车流量	常规模式：每1分钟上报一次
0.7 ~ 1.0	检测到异常事件	高频模式：每10秒上报一次，并附带语义摘要

自适应锚定的核心价值是**在同等链上资源消耗下，将保护力度集中在最需要的时段**，相比固定频率方案，在不增加成本的前提下，关键时段的保护粒度可提升 5~10 倍。

三、聚合网关层：批量压缩与跨设备聚合

聚合网关部署在区域机房中，负责收集辖区内所有边缘设备的 SegmentRoot，进一步构建**跨设备的 EpochMerkleTree**：

```
EpochRoot (每 30 秒，覆盖所有设备)
      /         |         \
Device₁_Root  Device₂_Root  Device₃_Root ...
```

每 30 秒向联盟链提交一次 EpochRoot。假设某区域有 1000 路摄像头，传统方案每分钟需要 1000 笔链上交易；而本方案通过两级聚合，每 30 秒只需 **1 笔交易**，链上存储开销降低了三个数量级。

聚合网关同时缓存完整的 Merkle 树结构和各设备上报的原始数据，供后续验证时提供 Merkle 证明路径。

四、联盟链层：存证与验证

4.1 链的选型

采用联盟链架构（如 Hyperledger Fabric 或 FISCO BCOS），参与节点包括：公安机关、司法鉴定机构、城市安防运营商、第三方审计机构等。联盟链相比公链的优势在于：可控的准入机制、确定性的交易终局性（无需等待多个区块确认）、更高的吞吐量（千级 TPS 足以满足需求）、无 Gas 费用负担。

4.2 智能合约设计

系统核心包含三个智能合约：

(A) 存证合约 AnchorContract

```
struct EpochAnchor {
    bytes32 epochRoot;       // 跨设备聚合 Merkle 根
    uint256 timestamp;       // 锚定时间戳
    uint256 epochId;         // 纪元编号
    address gateway;         // 提交网关地址
    uint16  deviceCount;     // 覆盖设备数量
    bytes32 semanticDigest;  // 语义摘要集合的哈希
```

```
}

function anchor(EpochAnchor calldata data) external onlyAutho
rizedGateway {
    require(data.timestamp > lastAnchorTime[msg.sender], "Tim
estamp regression");
    anchors[data.epochId] = data;
    lastAnchorTime[msg.sender] = data.timestamp;
    emit Anchored(data.epochId, data.epochRoot, data.timestam
p);
}
```

(B) 验证合约 VerifyContract

```
struct VerifyRequest {
    uint256 epochId;          // 待验证的纪元
    uint256 deviceIndex;      // 设备在 Epoch 树中的位置
    uint256 gopIndex;         // GOP 在设备 Segment 树中的位置
    bytes32 gopDualHash;      // 待验证 GOP 的双哈希值
    bytes32[] merkleProof;    // 从叶子到 EpochRoot 的完整 Merk
le 路径
}

function verify(VerifyRequest calldata req) external view ret
urns (uint8 status) {
    bytes32 storedRoot = anchors[req.epochId].epochRoot;
    bool valid = MerkleLib.verify(req.merkleProof, storedRoo
t, req.gopDualHash);

    if (valid) return 0;  // INTACT: 完全未篡改
    // 若密码学哈希不匹配但感知哈希匹配，返回不同状态码
    // ... 三态判定逻辑
    return 2;             // TAMPERED: 内容已被篡改
}
```

(C) 审计合约 AuditContract

负责记录所有验证请求和结果，形成不可篡改的审计日志。当验证结果为"已篡改"时，自动触发链上事件通知相关监管节点。

五、核心流程

5.1 存证流程（实时运行）

1. 摄像头 → 视频流 → 边缘 AI 盒子
2. 按 GOP 切分视频流
3. 对每个 GOP 计算 SHA-256 + 感知哈希
4. 运行 YOLOv8-nano 提取语义摘要
5. 计算 EIS，决定当前锚定频率
6. 构建三级 Merkle 树
7. 将视频分片写入 IPFS，获取 CID
8. 上报 SegmentRoot + 语义摘要 + CID 映射 → 聚合网关
9. 聚合为 EpochRoot
10. 提交上链 → 联盟链

5.2 验证流程（按需触发）

1. 验证请求方 → 指定[设备ID + 时间范围] → 验证服务
2. 从 IPFS 拉取原始视频分片
3. 重新计算目标 GOP 的双哈希
4. 从聚合网关获取 Merkle 路径
5. 调用链上 VerifyContract
6. 返回三态验证结果：
    - **INTACT**（完整）
    - **RE-ENCODED**（转码但内容未变）
    - **TAMPERED**（已篡改，定位到具体 GOP）

如果验证结果为 TAMPERED，验证服务进一步利用 Merkle 路径进行**二分定位**，逐层下探找到第一个不一致的叶子节点，输出精确的篡改时间范围（精度到 1~2 秒）。